



資料結構

Data Structure

Lab 7

姓名： 陳冠豪

學號： 114310125

Lab07-Q1

Convert Infix Expression to Postfix Using a Stack Implemented with Linked List

Code

```
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

//Stack 節點結構用 linked list 做)
struct Node {
    char data;    //存字 (運算子或括號)
    Node* next;  //指到下一個節點
};

// 用 linked list 實作 Stack
class Stack {
private:
    Node* top; //指向最上面那個
public:
    Stack() { top = nullptr; } //一開始是空的

    // push : 把東西放進去
    void push(char ch) {
        Node* newNode = new Node; //建新節點
        newNode->data = ch;        //存資料
        newNode->next = top;       //接到原本的 top
        top = newNode;            //更新 top
    }

    //pop: 把最上面的拿掉並回傳
    char pop() {
        if (isEmpty()) return '\0'; //空的就回傳空字元
        char ch = top->data;        //先存要回傳的值
        Node* temp = top;          //暫存目前 top
    }
};
```

```

        top = top->next;           //top 往下移
        delete temp;              //釋放記憶體
        return ch;
    }

    //peek: 只看最上面的值，不拿掉
    char peek() {
        return (top ? top->data : '\0');
    }

    //判斷是不是空 stack
    bool isEmpty() {
        return top == nullptr;
    }
};

//判斷運算子的優先順序
int precedence(char op) {
    //乘除優先
    if (op == '*' || op == '/') {
        return 2;
    }
    //加減比較低
    if (op == '+' || op == '-') {
        return 1;
    }
    //其他（像括號）就當最低
    return 0;
}

//中序轉後序
void InfixToPostfix(const char* infix, char* postfix) {
    Stack s;
    int j = 0; //postfix 的位置

    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];

```

```

//如果是數字或字母，直接輸出
if (isalnum(ch)) {
    postfix[j++] = ch;
}
//左括號直接丟進 stack
else if (ch == '(') {
    s.push(ch);
}
//右括號就一直 pop 到遇到左括號
else if (ch == ')') {
    while (!s.isEmpty() && s.peek() != '(') {
        postfix[j++] = s.pop();
    }
    s.pop(); //把'(' 丟（不用輸出）
}
//運算子處理
else {
    //如果 stack 裡面有比較高或一樣優先的，就先拿出來
    while (!s.isEmpty() && precedence(s.peek()) >= precedence(ch)) {
        postfix[j++] = s.pop();
    }
    s.push(ch); //最後再把自己放進去
}
}

//剩下的全部拿出來
while (!s.isEmpty()) {
    postfix[j++] = s.pop();
}

postfix[j] = '\0'; //字串結尾
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
    cin >> infix; //輸入中序式
}

```

```

InfixToPostfix(infix, postfix); //轉換
cout << "Postfix expression: " << postfix << endl; //印結果

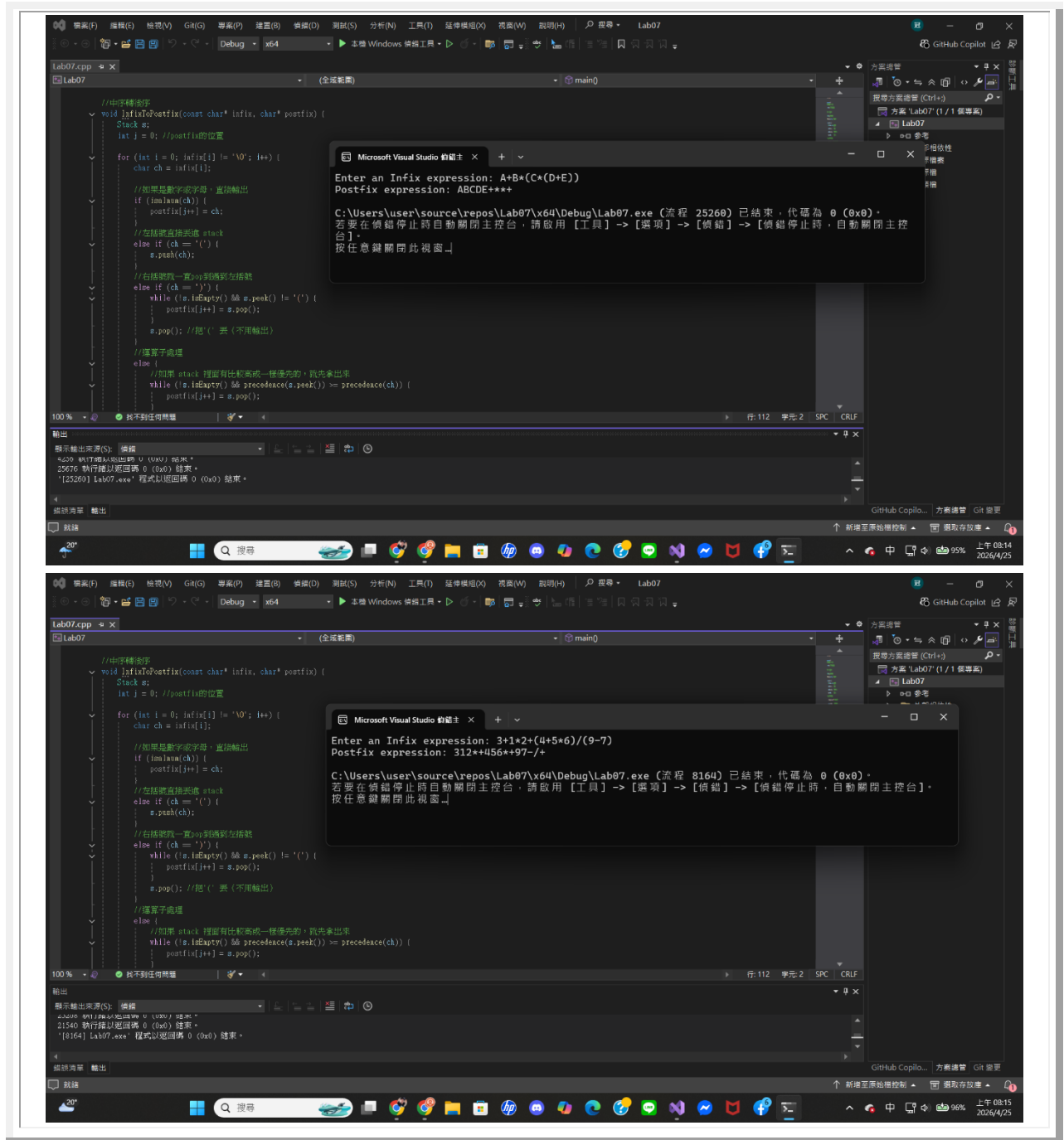
return 0;
}

```

Result

The screenshot displays the Microsoft Visual Studio IDE with the following components:

- Code Editor:** Shows the implementation of the `InfixToPostfix` function. It uses a stack to process the infix expression `(A+B)*D+E/(F+A*D)+C` and outputs the postfix expression `AB+D*EFAD**+/+C`.
- Output Window:** Displays the program's execution results, including the input infix expression, the resulting postfix expression, and a confirmation that the program completed successfully with a return value of 0.
- Taskbar:** Shows the Windows taskbar at the bottom with various application icons and the system clock indicating the time as 08:13 on 2024/4/25.



Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
(	(					0	

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
A	(					0	A
+	(	+				1	A
B	(	+				1	AB
)						-1	AB+
*	*					0	AB+
D	*					0	AB+D
+	+					0	AB+D*
E	+					0	AB+D*E
/	+	/				1	AB+D*E
(	+	/	(			2	AB+D*E
F	+	/	(			2	AB+D*EF

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
+	+	/	(	+		3	AB+D*EF
A	+	/	(	+		3	AB+D*EFA
*	+	/	(	*	+	4	AB+D*EFA
D	+	/	(	*	+	4	AB+D*EFAD
)	+	/				1	AB+D*EFA D*+
+	+	+				1	AB+D*EFA D*+/
C	+	+				1	AB+D*EFA D*+/C
<eos>						-1	AB+D*EFA D*+/C+

Token	Stack						Top	Output
	[0]	[1]	[2]	[3]	[4]	[5]		
A							-1	A
+	+						0	A
B	+						0	AB
*	+	*					1	AB
(	+	*	(				2	AB
C	+	*	(				2	ABC
*	+	*	(	*			3	ABC
(	+	*	(	*	(		4	ABC
D	+	*	(	*	(		4	ABCD
+	+	*	(	*	(	+	5	ABCD
E	+	*	(	*	(	+	5	ABCDE

Token	Stack						Top	Output
	[0]	[1]	[2]	[3]	[4]	[5]		
)	+	*	(	*			2	ABCDE+
)	+	*					1	ABCDE+*
<eos>							-1	ABCDE+** +

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
3						-1	3
+	+					0	3
1	+					0	31
*	+	*				1	31
2	+	*				1	312

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
+	+					0	312*+
(	+	(				1	312*+
4	+	(				1	312*+4
+	+	(	+			2	312*+4
5	+	(	+			2	312*+45
*	+	(	+	*		3	312*+45
6	+	(	+	*		3	312*+456
)	+					0	312*+456* +
/	+	/				1	312*+456* +
(	+	/	(			2	312*+456* +
9	+	/	(			2	312*+456* +9

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
-	+	/	(	-		3	312*+456* +9
7	+	/	(	-		3	312*+456* +97
)	+	/				1	312*+456* +97-
<eos>						-1	312*+456* +97-/+